

BANK ACCOUNT MANAGEMENT SYSTEM

Python Programming Project Statement

Project Type	First-Year Python Project
Programming Language	Python
Application Type	Menu-Driven Banking Simulation
Development Platform	Google Colab / Python IDE

Project Summary

The Bank Account Management System is a beginner-friendly Python application developed to simulate common banking operations. The program allows users to create accounts, check balances, deposit money, withdraw money, and display stored account information through a simple menu-driven interface.

The project focuses on applying fundamental Python programming concepts to a real-life problem. It does not connect to a real bank, database, or payment system; it is intended only for academic learning and demonstration.

Submitted as a Python Programming Project

1. Introduction

Banking applications are a common example of computer-based information systems. They manage customer information and perform operations such as deposits, withdrawals and balance enquiries. This project creates a simplified version of such a system using Python.

The program is designed around a menu. The user selects an operation, provides the required information, and receives an appropriate result. After completing an operation, the menu appears again until the user chooses to exit.

2. Problem Statement

Design and implement a Python program that can manage basic bank account information and perform common account operations. The system should provide a simple interface through which a user can create an account, view account details, deposit money, withdraw money, and display all available accounts.

3. Objectives

- To develop a simple real-life application using Python.
- To understand how dictionaries can store structured account information.
- To use functions to divide the program into manageable modules.
- To apply conditional statements for decision making and validation.
- To use loops for repeated menu operations.
- To perform basic arithmetic operations for deposits and withdrawals.
- To understand how user input can be processed in a menu-driven program.

4. Scope of the Project

The current project is limited to basic banking operations and in-memory data storage. It is suitable for academic demonstration and learning. It can later be extended with file handling, databases, authentication, transaction history and a graphical user interface.

5. System Features and Working

5.1 Create Account

The user enters an account number, account holder name and initial deposit. Before creating the account, the program checks whether the account number already exists. If it exists, a duplicate account is not created.

5.2 Check Balance

The user enters an account number. If the account is present, the program displays the account holder's name and current balance. Otherwise, it displays an account-not-found message.

5.3 Deposit Money

For a valid account, the user enters a deposit amount. A positive amount is added to the existing balance. The updated balance is then displayed.

5.4 Withdraw Money

The user enters the account number and withdrawal amount. The program verifies that the amount is positive and that sufficient balance is available. Only then is the amount subtracted from the account balance.

5.5 Display All Accounts

The system can display all currently stored accounts. For each account, the account number, account holder name and balance are shown.

6. Program Flow

Step	Operation
1	Start the program
2	Display the main menu
3	Accept the user's choice
4	Perform the selected banking operation
5	Validate account and amount where required
6	Display the result
7	Return to the main menu
8	Exit when the user selects Exit

Main Menu

1. Create Account 2. Check Balance 3. Deposit Money 4. Withdraw Money 5. Display All Accounts 6. Exit

7. Python Concepts Used

Functions

Separate functions are used for creating accounts, checking balances, depositing money, withdrawing money and displaying accounts. This makes the program easier to read and maintain.

Dictionary

A dictionary named 'accounts' stores account numbers as keys. Each account contains the account holder's name and balance as values. Dictionaries provide a convenient way to organize related information.

Conditional Statements

if, elif and else statements are used to make decisions, such as checking whether an account exists, validating amounts, and checking sufficient balance.

Loops

A while loop keeps the menu running until the user chooses the Exit option. A for loop is used to display all accounts stored in the dictionary.

Input and Arithmetic Operations

The input() function accepts information from the user. Arithmetic operators are used to add deposits to the balance and subtract withdrawals from it.

8. Sample Test Cases

Test	Input	Expected Result
Create	101, Harsh, 5000	Account created
Deposit	101, 2000	Balance becomes 7000
Withdraw	101, 1000	Balance becomes 6000
Balance	101	Displays Rs. 6000
Invalid withdrawal	101, 10000	Insufficient balance
Duplicate account	101	Account already exists

9. Advantages

- Simple and beginner-friendly interface.
- Uses important Python programming concepts.
- Easy to modify and expand.
- Demonstrates practical use of functions and dictionaries.
- Provides basic validation for common operations.

10. Limitations

- Data is stored only while the program is running.
- No database is connected.
- There is no real authentication or PIN system.
- The application cannot perform real financial transactions.
- The project is intended for educational use only.

11. Future Enhancements

- Add PIN or password-based authentication.
- Add transaction history and account statements.
- Add money transfer between accounts.
- Store information permanently using files or SQLite/MySQL.
- Add account update and deletion options.
- Add interest calculation and monthly statements.
- Develop a graphical user interface using Tkinter.
- Improve validation and security.

12. Learning Outcomes

This project provides practical experience in converting a real-life requirement into a structured Python program. It improves understanding of functions, dictionaries, loops, conditional statements, input handling, validation and basic problem solving.

13. Conclusion

The Bank Account Management System successfully demonstrates how fundamental Python programming concepts can be combined to create a useful menu-driven application. The system performs basic account operations such as account creation, balance enquiry, deposit, withdrawal and account display.

Although the project is a simplified simulation and does not provide the security or persistence required by a real banking application, it provides a strong foundation for future development. Additional features such as databases, authentication, transaction history and GUI support can transform the project into a more advanced application.

Project Status: Completed – Educational Python Simulation

End of Project Statement